

M7 EVALUACIÓN MODULAR

Romina Angel

Profesor: César Arnaez

API RESTful PELÍCULAS Y ACTORES

Node.js + Express + PostgreSQL + Sequelize
Aplicación Backend con Arquitectura RESTful

```
1 const express = require('express');
2 const app = express();
3 const { Pelicula, Actor } = require('./models');
4 app.use(express.json());
5
6 // Rutas
7 app.get('/peliculas', async (req, res) => {
8   const peliculas = await Pelicula.findAll();
9   res.json(peliculas);
10 });
11
12 app.post('/peliculas', async (req, res) => {
13   const pelicula = await Pelicula.create(req.body);
14   res.status(201).json(pelicula);
15 });
16
17 app.listen(3000, () => {
18   console.log('Servidor corriendo en puerto 3000');
19 });
```

Node.js
Entorno de ejecución JavaScript

Express
Framework web para Node.js

Sequelize
ORM para PostgreSQL

PostgreSQL
Base de datos relacional

Postman
Pruebas de API

Pelicula

- id (PK)
- titulo
- genero
- anio

PeliculaActor

- peliculaId (FK)
- actorId (FK)

Actor

- id (PK)
- nombre
- edad
- nacionalidad

CRUD - API Endpoints

GET	/peliculas	Obtener todas las peliculas
POST	/peliculas	Crear una pelicula
PUT	/peliculas/:id	Actualizar una pelicula
DELETE	/peliculas/:id	Eliminar una pelicula
GET	/actores	Obtener todos los actores
POST	/actores	Crear un actor
PUT	/actores/:id	Actualizar un actor
DELETE	/actores/:id	Eliminar un actor
POST	/pelicula-actor	Relacionar actor con pelicula

Postman - Ejemplo de Prueba

POST http://localhost:3000/pelicula-actor

Body - raw - JSON

```
{
  "peliculaId": 1,
  "actorId": 1
}
```

Response 201 Created

```
{
  "mensaje":
    "Actor agregado a película"
}
```

TITULO _____
GENERO _____
AÑO _____
ACTORES _____

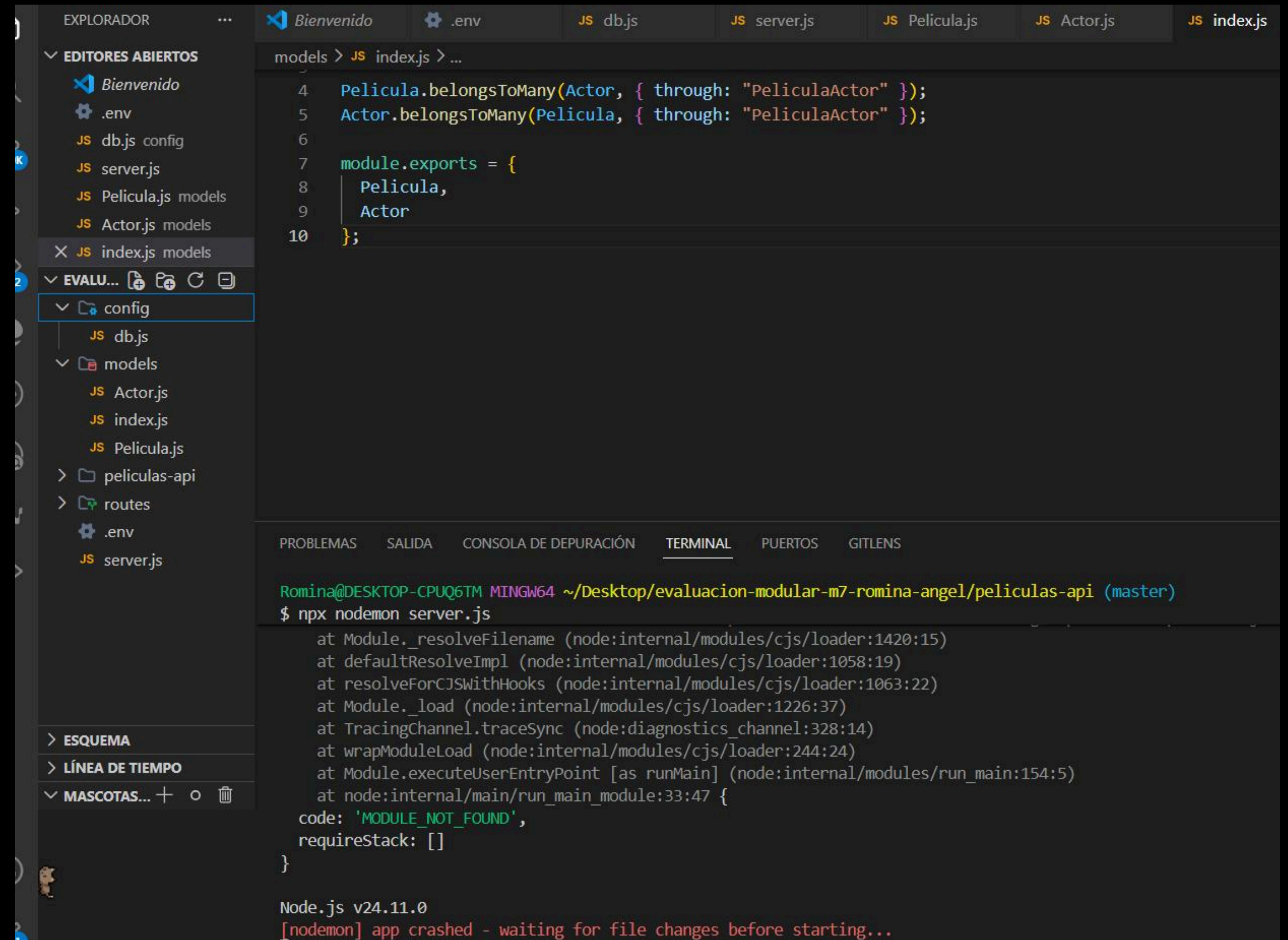
Objetivo del Proyecto

- **Desarrollar una API REST utilizando Node.js, Express, PostgreSQL y Sequelize para administrar películas y actores.**

Tecnologías Utilizadas

- Node.js
- Express
- PostgreSQL
- Sequelize
- Postman
- VS Code

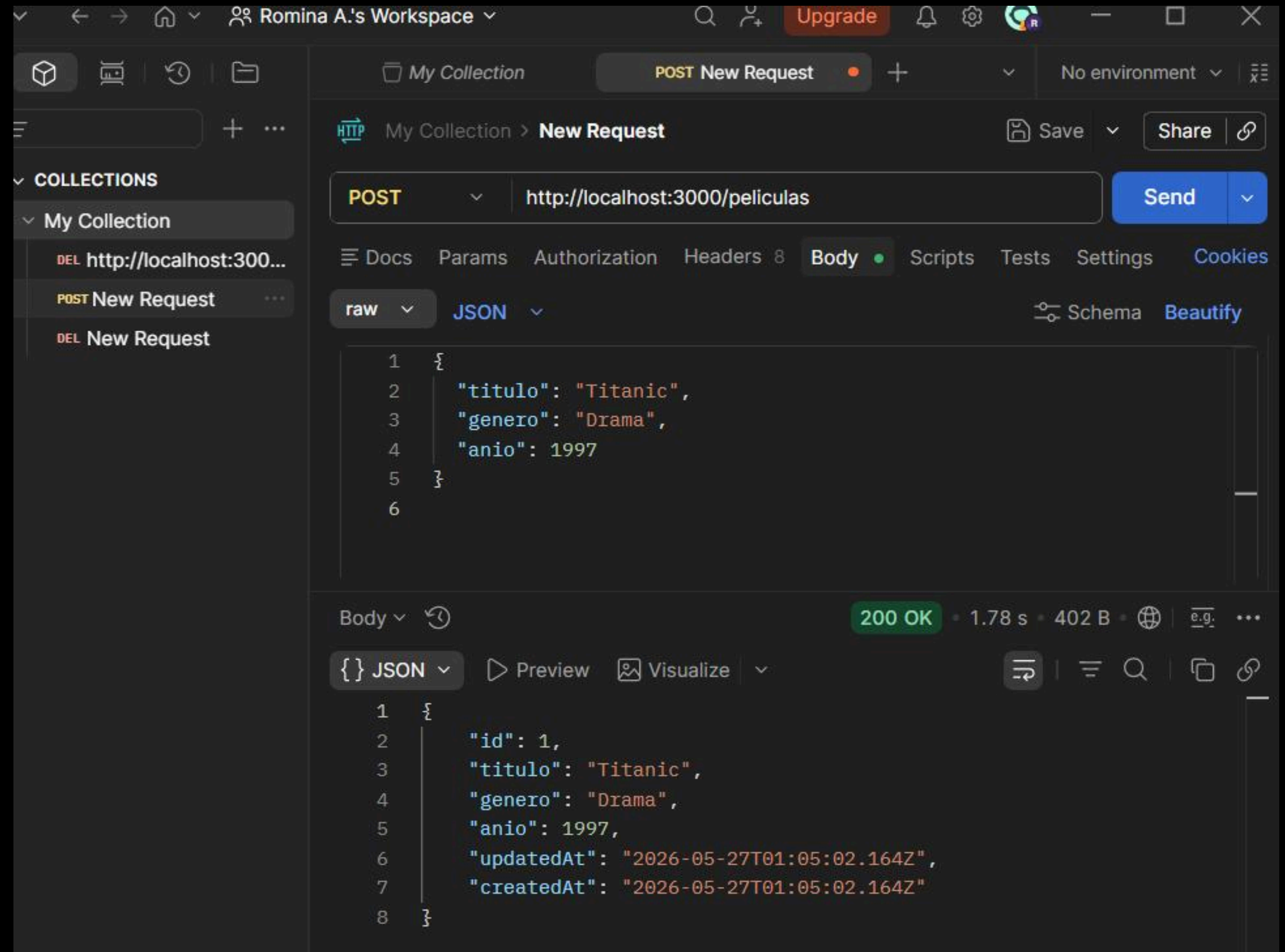
```
PS C:\Users\Romina\Desktop\evaluacion-modular-m7-romina-angel> cd ..
PS C:\Users\Romina\Desktop> npx nodemon server.js
Need to install the following packages:
nodemon@3.1.14
Ok to proceed? (y)
```



MÉTODOS HTTP

Métodos implementados:

- GET → obtener datos
- POST → crear datos
- PUT → actualizar datos
- DELETE → eliminar datos



⌵

⬅ ➡ 🏠 ⌵

Romina A.'s Workspace ⌵

🔍 👤 Upgrade 🔔 ⚙️ 🔄

— □ ✕

🏠 📄 🔄 📁

⌵

+

...

COLLECTIONS

⌵ My Collection

DEL http://localhost:300...

POST New Request

DEL New Request

My Collection

GET New Request

+

⌵

No environment ⌵

⌵

HTTP

My Collection > New Request

Save

Share

🔗

GET

⌵

http://localhost:3000/peliculas

Send

⌵

Docs

Params

Authorization

Headers 8

Body ●

Scripts

Tests

Settings

Cookies

raw

⌵

JSON

⌵

Schema

Beautify

1 {

2 "titulo": "Titanic",

3 "genero": "Drama",

4 "anio": 1997

5 }

6

Body

⌵

🔄

200 OK

80 ms

404 B

🌐

e.g.

...

{ } JSON

⌵

Preview

Visualize

⌵

🔄

≡

🔍

📄

🔗

1 [

2 {

3 "id": 1,

4 "titulo": "Titanic",

5 "genero": "Drama",

6 "anio": 1997,

7 "createdAt": "2026-05-27T01:05:02.164Z",

8 "updatedAt": "2026-05-27T01:05:02.164Z"

9 }

10]

⌵

⬅ ➡ 🏠 ⌵

⌵

🔍 👤 Upgrade 🔔 ⚙️ 🔄

— □ ✕

🏠 📄 🔄 📁

⌵

+

...

COLLECTIONS

⌵ My Collection

DEL http://localhost:300...

POST New Request

DEL New Request

My Collection

PUT New Request

+

⌵

No environment ⌵

⌵

HTTP

My Collection > New Request

Save

Share

🔗

PUT

⌵

http://localhost:3000/peliculas/1

Send

⌵

Docs

Params

Authorization

Headers 8

Body ●

Scripts

Tests

Settings

Cookies

raw

⌵

JSON

⌵

Schema

Beautify

1 {

2 "titulo": "Titanic 2",

3 "genero": "Drama",

4 "anio": 2000

5 }

Body

⌵

🔄

200 OK

169 ms

302 B

🌐

e.g.

...

{ } JSON

⌵

Preview

Visualize

⌵

🔄

≡

🔍

📄

🔗

1 {

2 "mensaje": "Película actualizada"

3 }

My Collection

GET New Request

No environment

My Collection > New Request

Save

Share

GET

http://localhost:3000/peliculas

Send

Docs

Params

Authorization

Headers 8

Body

Scripts

Tests

Settings

Cookies

raw

JSON

Schema

Beautify

```
1 {
2   "titulo": "Titanic 2",
3   "genero": "Drama",
4   "anio": 2000
5 }
```

Body

200 OK

52 ms

406 B

Preview

Visualize

```
1 [
2   {
3     "id": 1,
4     "titulo": "Titanic 2",
5     "genero": "Drama",
6     "anio": 2000,
7     "createdAt": "2026-05-27T01:05:02.164Z",
8     "updatedAt": "2026-05-27T01:13:03.914Z"
9   }
]
```

My Collection

POST New Request

No environment

My Collection > New Request

Save

Share

POST

http://localhost:3000/pelicula-actor

Send

Docs

Params

Authorization

Headers 8

Body

Scripts

Tests

Settings

Cookies

raw

JSON

Schema

Beautify

```
1 {
2   "peliculaId": 1,
3   "actorId": 1
4 }
```

Body

200 OK

103 ms

307 B

Preview

Visualize

```
1 {
2   "mensaje": "Actor agregado a pelicula"
3 }
```

**[https://github.com/
PiercetheRomi/
Evaluacion-modular-
modulo-7.git](https://github.com/PiercetheRomi/Evaluacion-modular-modulo-7.git)**

CÓDIGOS HTTP

Modelo Película:

- título
- género
- año
-

Modelo Actor:

- nombre
- edad
- nacionalidad